

Abstract / purpose and design

BerlinBite is a cloud-programming prototype for restaurant discovery in Berlin. It addresses a small but recognizable planning problem: users can find a restaurant, yet still need to confirm which branch they mean, inspect opening information and decide whether to contact the venue before visiting. The intended interaction is deliberately short: search, select, review and save. Its audience includes visitors and residents; an English-speaking visitor meeting a friend is the primary design persona.

The implementation uses FastAPI and a lightweight HTML, CSS and JavaScript frontend. The same application serves both browser assets and API routes. A Docker image packages these components together, allowing local and cloud execution without an AWS-specific rewrite. Provider adapters isolate Google Places, current weather and optional AI advice. Configuration is supplied through environment variables and private secrets rather than source changes.

The application distinguishes facts from estimates. Google Places supplies restaurant details and, where available, editorial text with attribution. The app does not fabricate restaurant results when access is unavailable. A deterministic time-of-day and weekday rule produces a clearly labelled crowd estimate. This is not a trained machine-learning model or measured occupancy. Staff language and reservation availability remain unknown unless supported by a reliable source; the user is directed to contact the restaurant.

OpenAI is used only for generic advice based on the time heuristic. Google reviews and editorial content are not sent to that service. Weather is labelled as current conditions and is not presented as a forecast for the chosen visit. Saved lists retain place identifiers in browser storage and retrieve details again when opened. This design avoids an unnecessary database and account system, while acknowledging that saved lists do not synchronize across devices.

Abstract / implementation and reflection

The proposed AWS deployment uses CloudFront as the HTTPS entry point, WAF for a request-rate rule, an Application Load Balancer and ECS Fargate tasks across two availability zones. ECR stores versioned container images. Secrets Manager holds provider keys, and CloudWatch collects operational logs and an error alarm. Terraform records the infrastructure so another authorized operator can review and recreate it.

The cloud design balances availability, clarity and cost. Two tasks form the baseline, with CPU-based scaling up to four. Health checks and deployment rollback are configured. Tasks receive outbound internet connectivity through public subnets, while their security group accepts inbound traffic only from the load balancer. This avoids NAT Gateway expense. The browser connection uses HTTPS, but the current CloudFront-to-load-balancer hop uses HTTP; adding an origin domain and certificate is a documented production improvement.

During preparation, the earlier App Runner suggestion was revisited because AWS documentation states that it is closed to new customers. Standard ECS was selected to preserve the portable container and make infrastructure choices explicit. ECS Express Mode, Lambda and a separate S3 frontend remain documented alternatives. The final choice should be reviewed against the actual deadline, required security guarantees and current pricing before deployment.

At this stage, source files and portfolio drafts exist. Python syntax, JavaScript syntax, Terraform formatting and all 12 API, adapter and scoring tests passed. The user installed dependencies and successfully started the local server. Docker build/runtime checks and Terraform provider validation also passed after approved command access became available. Real Google search passed. The user reviewed the local UI. AWS deployment completed and the HTTPS endpoint passed homepage, search, details and AI insight checks. The next phase must collect real evidence, resolve failures and update these documents accordingly.

OpenAI Codex assisted with drafting code and documentation. The student remains responsible for understanding the implementation, testing it, explaining the tradeoffs and complying with institutional AI-use guidance. No tutor feedback, deployment results or performance measurements have been invented.